



Implement a robust Java calculator program that captures user input dynamically, processes mathematical operations using conditional logic and looping constructs, and ensures efficient error handling

```
import java.util.Scanner;
public class Calculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean continueCalc = true;
        System.out.println("===== JAVA CONSOLE CALCULATOR =====");
        while (continueCalc) {
            try {
                // Take first number
                System.out.print("Enter first number: ");
                double num1 = Double.parseDouble(scanner.nextLine());

                // Take operator
                System.out.print("Enter operator (+, -, *, /, %): ");
                char operator = scanner.nextLine().charAt(0);

                // Take second number
                System.out.print("Enter second number: ");
                double num2 = Double.parseDouble(scanner.nextLine());
                double result;
                // Conditional logic for operation
                switch (operator) {
                    case '+':
                        result = num1 + num2;
                        System.out.println("Result: " + result);
                        break;
                    case '-':
                        result = num1 - num2;
                        System.out.println("Result: " + result);
                        break;
                    case '*':
                        result = num1 * num2;
                        System.out.println("Result: " + result);
                        break;
                    case '/':
                        if (num2 == 0) {
                            System.out.println("Error: Cannot divide by zero.");
                        } else {
                            result = num1 / num2;
                            System.out.println("Result: " + result);
                        }
                        break;
                    case '%':
                        if (num2 == 0) {
                            System.out.println("Error: Cannot modulo by zero.");
                        } else {
                            result = num1 % num2;
                        }
                }
            } catch (Exception e) {
                System.out.println("Invalid input. Please enter a valid number or operator.");
            }
        }
    }
}
```



```
        System.out.println("Result: " + result);
    }
    break;
default:
    System.out.println("Error: Invalid operator.");
}

} catch (NumberFormatException e) {
    System.out.println("Error: Invalid number input.");
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}

// Ask user if they want to continue
System.out.print("Do you want to perform another calculation? (yes/no): ");
String response = scanner.nextLine().trim().toLowerCase();
if (!response.equals("yes")) {
    continueCalc = false;
}
}

System.out.println("Thank you for using the calculator. Goodbye!");
scanner.close();
}
}
```



Develop a Java program for an E-commerce order processing where some products are initialized through multiple constructors, overloaded constructors, where users can input some product details manually, the system computes total order cost dynamically, applies discount policies based on conditions, and presents a detailed invoice summarizing the purchase.

```
import java.util.Scanner;
class Product {
    String name;
    double price;
    int quantity;
    // Default constructor
    public Product() {
        this.name = "Unknown";
        this.price = 0;
        this.quantity = 0;
    }

    // Overloaded constructor 1
    public Product(String name, double price) {
        this.name = name;
        this.price = price;
        this.quantity = 1;
    }

    // Overloaded constructor 2
    public Product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    double getTotalCost() {
        return price * quantity;
    }

    void displayProduct() {
        System.out.printf("%-20s %-10.2f %-10d %-10.2f\n", name, price, quantity, getTotalCost());
    }
}

public class ECommerceOrderProcessor {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Product[] cart = new Product[10];
        int count = 0;
        System.out.println("==== E-COMMERCE ORDER PROCESSING ==== \n");
        // Predefined Products
        cart[count++] = new Product("Laptop", 45000, 1);
        cart[count++] = new Product("Mouse", 500, 2);

        // Input additional products manually
        System.out.print("How many more products do you want to add? ");
```



```
int more = Integer.parseInt(scanner.nextLine());
for (int i = 0; i < more; i++) {
    System.out.println("\nEnter details for product " + (i + 1) + ":");
    System.out.print("Product Name: ");
    String name = scanner.nextLine();
    System.out.print("Price: ");
    double price = Double.parseDouble(scanner.nextLine());
    System.out.print("Quantity: ");
    int quantity = Integer.parseInt(scanner.nextLine());

    cart[count++] = new Product(name, price, quantity);
}
// Print Invoice
double total = 0;
System.out.println("\n===== INVOICE =====");
System.out.printf("%-20s %-10s %-10s %-10s\n", "Product", "Price", "Qty", "Subtotal");
System.out.println("-----");
for (int i = 0; i < count; i++) {
    cart[i].displayProduct();
    total += cart[i].getTotalCost();
}
// Apply discount
double discount = 0;
if (total > 5000) {
    discount = total * 0.10; // 10% off
} else if (total > 2000) {
    discount = total * 0.05; // 5% off
}
double finalTotal = total - discount;
System.out.println("-----");
System.out.printf("%-42s: ₹%.2f\n", "Total", total);
System.out.printf("%-42s: ₹%.2f\n", "Discount", discount);
System.out.printf("%-42s: ₹%.2f\n", "Final Amount Payable", finalTotal);
System.out.println("\nThank you for shopping with us!");
scanner.close();
}
```

==== E-COMMERCE ORDER PROCESSING ====

How many more products do you want to add? 2

Enter details for product 1:

Product Name: Keyboard

Price: 1500

Quantity: 1

Enter details for product 2:

Product Name: Monitor

Price: 8000

Quantity: 1

===== INVOICE =====

Product	Price	Qty	Subtotal
---------	-------	-----	----------



KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)



Laptop	45000.00	1	45000.00
Mouse	500.00	2	1000.00
Keyboard	1500.00	1	1500.00
Monitor	8000.00	1	8000.00

Total	:	₹55500.00
Discount	:	₹5550.00
Final Amount Payable	:	₹49950.00

Thank you for shopping with us!



Develop a Java program that implements a simple hotel room booking system using twodimensional arrays. The system allows users to: View available and booked rooms, Book a room by selecting a floor and room number and exit the system when finished

```
import java.util.Scanner;
public class HotelBookingSystem {
    public static void main(String[] args) {
        final int FLOORS = 5;
        final int ROOMS_PER_FLOOR = 4;
        boolean[][] rooms = new boolean[FLOORS][ROOMS_PER_FLOOR]; // false = available, true = booked
        Scanner scanner = new Scanner(System.in);
        int choice;

        System.out.println("==== HOTEL ROOM BOOKING SYSTEM =====");

        do {
            System.out.println("\n1. View Rooms");
            System.out.println("2. Book a Room");
            System.out.println("3. Exit");
            System.out.print("Enter your choice: ");
            choice = Integer.parseInt(scanner.nextLine());

            switch (choice) {
                case 1:
                    System.out.println("\nRoom Status (F = Free, B = Booked):");
                    for (int i = 0; i < FLOORS; i++) {
                        System.out.print("Floor " + (i + 1) + ": ");
                        for (int j = 0; j < ROOMS_PER_FLOOR; j++) {
                            System.out.print(rooms[i][j] ? "B " : "F ");
                        }
                        System.out.println();
                    }
                    break;

                case 2:
                    System.out.print("Enter floor number (1-" + FLOORS + "): ");
                    int floor = Integer.parseInt(scanner.nextLine()) - 1;
                    System.out.print("Enter room number (1-" + ROOMS_PER_FLOOR + "): ");
                    int room = Integer.parseInt(scanner.nextLine()) - 1;

                    if (floor < 0 || floor >= FLOORS || room < 0 || room >= ROOMS_PER_FLOOR) {
                        System.out.println("Invalid floor or room number!");
                    } else if (rooms[floor][room]) {
                        System.out.println("Room is already booked.");
                    } else {
                        rooms[floor][room] = true;
                        System.out.println("Room booked successfully!");
                    }
                    break;

                case 3:
                    System.out.println("Exiting the system. Thank you!");
            }
        } while (choice != 3);
    }
}
```



```
        break;

        default:
            System.out.println("Invalid choice. Try again.");
    }

    } while (choice != 3);

    scanner.close();
}
}
```

Floor 1: F F B F

Floor 2: F F F F

...

Create a Java program demonstrating single inheritance where a subclass extends a superclass and calls its methods

```
// Superclass
class Person {
    String name;
    int age;
    void setDetails(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Subclass
class Employee extends Person {
    String designation;
    void setEmployeeDetails(String name, int age, String designation) {
        // Call superclass method
        setDetails(name, age);
        this.designation = designation;
    }
    void displayEmployeeDetails() {
        // Call superclass method
        displayDetails();
        System.out.println("Designation: " + designation);
    }
}

// Main class
public class InheritanceDemo {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.setEmployeeDetails("Alice", 28, "Software Engineer");
        System.out.println("=== Employee Details ===");
    }
}
```



```
        emp.displayEmployeeDetails();
    }
}
=== Employee Details ===
Name: Alice
Age: 28
Designation: Software Engineer
```

Implement Multiple Inheritance using interface in Java to demonstrate polymorphism.

```
// Interface 1
interface Printable {
    void print();
}
// Interface 2
interface Showable {
    void show();
}
// Class implementing both interfaces
class Document implements Printable, Showable {
    @Override
    public void print() {
        System.out.println("Printing the document...");
    }

    @Override
    public void show() {
        System.out.println("Showing the document on screen...");
    }
}
// Main class
public class InterfaceInheritanceDemo {
    public static void main(String[] args) {
        // Polymorphism: Using interface reference
        Printable p = new Document();
        Showable s = new Document();
        System.out.println("=== Using Printable interface reference ===");
        p.print();
        System.out.println("\n=== Using Showable interface reference ===");
        s.show();
        System.out.println("\n=== Using actual Document object ===");
        Document doc = new Document();
        doc.print();
        doc.show();
    }
}
=== Using Printable interface reference ===
Printing the document...
=== Using Showable interface reference ===
Showing the document on screen...
=== Using actual Document object ===
Printing the document...
Showing          the          document          on          screen...
```




Develop a Java program for simulation of any real time application with required functionalities. For eg. ATM machine with functionalities like checking account balance, withdrawing, and depositing money. Use try, catch, and finally blocks to handle potential exceptions such as insufficient funds (throwing ArithmeticException) and invalid input (throwing IllegalArgumentException). Ensure that the application continues to run smoothly after handling exceptions.

```
import java.util.Scanner;
public class ATMSimulator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double balance = 5000.00; // Initial balance
        int choice;
        System.out.println("=== Welcome to the Java ATM Machine ===");
        while (true) {
            System.out.println("\n1. Check Balance");
            System.out.println("2. Deposit Money");
            System.out.println("3. Withdraw Money");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

            try {
                choice = Integer.parseInt(scanner.nextLine());

                switch (choice) {
                    case 1:
                        System.out.printf("Your current balance is ₹%.2f\n", balance);
                        break;

                    case 2:
                        System.out.print("Enter amount to deposit: ");
                        double deposit = Double.parseDouble(scanner.nextLine());

                        if (deposit <= 0) {
                            throw new IllegalArgumentException("Amount must be positive.");
                        }

                        balance += deposit;
                        System.out.println("Deposit successful!");
                        break;

                    case 3:
                        System.out.print("Enter amount to withdraw: ");
                        double withdraw = Double.parseDouble(scanner.nextLine());

                        if (withdraw <= 0) {
                            throw new IllegalArgumentException("Amount must be positive.");
                        }
                        if (withdraw > balance) {
                            throw new ArithmeticException("Insufficient funds.");
                        }

```



```
        }

        balance -= withdraw;
        System.out.println("Withdrawal successful!");
        break;

    case 4:
        System.out.println("Thank you for using the ATM. Goodbye!");
        scanner.close();
        return;

    default:
        System.out.println("Invalid choice. Please select between 1 and 4.");
    }

    } catch (NumberFormatException e) {
        System.out.println("Invalid input! Please enter numeric values.");
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (ArithmeticException e) {
        System.out.println("Error: " + e.getMessage());
    } finally {
        System.out.println("Returning to main menu...\n");
    }
    }
}

=== Welcome to the Java ATM Machine ===
```

```
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 3
Enter amount to withdraw: 10000
Error: Insufficient funds.
Returning to main menu...
```



Create a multi-threaded Java application that simulates any real time application with required functionalities. For eg. Basic chat system in which each user (thread) sends and receives messages. Use `isAlive()` to check the status of threads and `join()` to ensure proper synchronization. Implement thread priorities to handle high-priority messages and demonstrate thread suspension, resumption, and stopping.

```
class ChatUser extends Thread {
    private final String username;
    private final String[] messages;
    private volatile boolean paused = false;
    private volatile boolean running = true;

    public ChatUser(String name, String[] messages) {
        this.username = name;
        this.messages = messages;
    }
    public void run() {
        try {
            for (String msg : messages) {
                while (paused) {
                    Thread.sleep(100); // Wait while paused
                }

                if (!running) break; // Exit if stopped
                System.out.println(username + ": " + msg);
                Thread.sleep(500); // Simulate delay between messages
            }
        } catch (InterruptedException e) {
            System.out.println(username + " was interrupted.");
        }
    }
    public void pauseChat() {
        paused = true;
    }
    public void resumeChat() {
        paused = false;
    }
    public void stopChat() {
        running = false;
    }
}

public class MultiThreadedChat {
    public static void main(String[] args) throws InterruptedException {
        String[] aliceMsgs = {"Hi Bob!", "How are you?", "What are you working on?"};
        String[] bobMsgs = {"Hey Alice!", "I'm good.", "Working on a Java project."};
        ChatUser alice = new ChatUser("Alice", aliceMsgs);
        ChatUser bob = new ChatUser("Bob", bobMsgs);
        // Set priorities (simulate high-priority messages from Alice)
        alice.setPriority(Thread.MAX_PRIORITY);
        bob.setPriority(Thread.NORM_PRIORITY);
    }
}
```



```
System.out.println("Chat is starting...\n");
alice.start();
bob.start();

// Wait for a while and then pause Bob
Thread.sleep(1000);
System.out.println("\n[System] Pausing Bob's chat temporarily...\n");
bob.pauseChat();

// Let Alice continue alone for some time
Thread.sleep(1500);
System.out.println("\n[System] Resuming Bob's chat...\n");
bob.resumeChat();

// Wait and then stop Alice early
Thread.sleep(1000);
System.out.println("\n[System] Stopping Alice's chat...\n");
alice.stopChat();

// Synchronize threads
alice.join();
bob.join();

// Check status
System.out.println("\n[System] Is Alice alive? " + alice.isAlive());
System.out.println("[System] Is Bob alive? " + bob.isAlive());

System.out.println("\nChat session ended.");
}
}
Chat is starting...

Alice: Hi Bob!
Bob: Hey Alice!
Alice: How are you?
Bob: I'm good.

[System] Pausing Bob's chat temporarily...

Alice: What are you working on?

[System] Resuming Bob's chat...

Bob: Working on a Java project.

[System] Stopping Alice's chat...

[System] Is Alice alive? false
[System] Is Bob alive? false

Chat session ended.
```



PART B

Write a C/C++/Java program to draw the following pattern using (a) the DDA line drawing algorithm for both rectangles with Dotted, Thick line style and (b) Bresenham's line drawing algorithm for a diamond shape with Dashed, Solid line style.

```
import javax.swing.*;
import java.awt.*;
import java.awt.image.BufferedImage;
public class LineDrawing extends JPanel {
    BufferedImage canvas;
    public LineDrawing() {
        canvas = new BufferedImage(800, 600, BufferedImage.TYPE_INT_RGB);
        Graphics2D g = canvas.createGraphics();
        g.setColor(Color.WHITE);
        g.fillRect(0, 0, 800, 600);
        // Draw Rectangles with DDA
        drawDDA(50, 50, 200, 50, "dotted", Color.BLUE);
        drawDDA(200, 50, 200, 200, "dotted", Color.BLUE);
        drawDDA(200, 200, 50, 200, "dotted", Color.BLUE);
        drawDDA(50, 200, 50, 50, "dotted", Color.BLUE);

        drawDDA(300, 50, 450, 50, "thick", Color.RED);
        drawDDA(450, 50, 450, 200, "thick", Color.RED);
        drawDDA(450, 200, 300, 200, "thick", Color.RED);
        drawDDA(300, 200, 300, 50, "thick", Color.RED);

        // Draw Diamond with Bresenham
        drawBresenham(150, 300, 250, 400, "dashed", Color.GREEN);
        drawBresenham(250, 400, 150, 500, "dashed", Color.GREEN);
        drawBresenham(150, 500, 50, 400, "dashed", Color.GREEN);
        drawBresenham(50, 400, 150, 300, "dashed", Color.GREEN);

        drawBresenham(400, 300, 500, 400, "solid", Color.MAGENTA);
        drawBresenham(500, 400, 400, 500, "solid", Color.MAGENTA);
        drawBresenham(400, 500, 300, 400, "solid", Color.MAGENTA);
        drawBresenham(300, 400, 400, 300, "solid", Color.MAGENTA);
    }

    public void drawDDA(int x1, int y1, int x2, int y2, String style, Color color) {
        Graphics2D g = canvas.createGraphics();
        g.setColor(color);

        int dx = x2 - x1;
        int dy = y2 - y1;

        int steps = Math.max(Math.abs(dx), Math.abs(dy));
        float xInc = dx / (float) steps;
        float yInc = dy / (float) steps;

        float x = x1;
```



```
float y = y1;

for (int i = 0; i <= steps; i++) {
    if (style.equals("dotted")) {
        if (i % 4 == 0) canvas.setRGB(Math.round(x), Math.round(y), color.getRGB());
    } else if (style.equals("thick")) {
        for (int j = -1; j <= 1; j++) {
            for (int k = -1; k <= 1; k++) {
                int px = Math.round(x) + j;
                int py = Math.round(y) + k;
                if (px >= 0 && py >= 0 && px < 800 && py < 600)
                    canvas.setRGB(px, py, color.getRGB());
            }
        }
    }
    x += xInc;
    y += yInc;
}

public void drawBresenham(int x1, int y1, int x2, int y2, String style, Color color) {
    Graphics2D g = canvas.createGraphics();
    g.setColor(color);

    int dx = Math.abs(x2 - x1);
    int dy = Math.abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    boolean isSteep = dy > dx;
    int err = (isSteep ? dy : dx) / 2;
    int count = 0;

    while (true) {
        boolean plot = true;

        if (style.equals("dashed")) {
            if ((count / 10) % 2 == 0) plot = true;
            else plot = false;
        } else if (style.equals("solid")) {
            plot = true;
        }

        if (plot) canvas.setRGB(x1, y1, color.getRGB());

        if (x1 == x2 && y1 == y2) break;

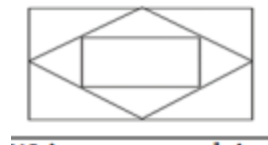
        if (isSteep) {
            y1 += sy;
            err -= dx;
            if (err < 0) {
                x1 += sx;
                err += dy;
            }
        }
    }
}
```



```
    } else {
        x1 += sx;
        err -= dy;
        if (err < 0) {
            y1 += sy;
            err += dx;
        }
    }
    count++;
}

protected void paintComponent(Graphics g) {
    g.drawImage(canvas, 0, 0, null);
}

public static void main(String[] args) {
    JFrame frame = new JFrame("Line Drawing Algorithms - DDA & Bresenham");
    LineDrawing panel = new LineDrawing();
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.add(panel);
    frame.setSize(820, 640);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
}
}
```





Write a menu driven program in C/C++/Java to draw circle using DDA, Bresenham's , Midpoint circle drawing algorithm with different styles as solid, dotted and dashed circles.

```
import javax.swing.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Scanner;
public class CircleDrawer extends JPanel {
    BufferedImage canvas;
    int centerX = 300, centerY = 300, radius = 100;
    String algorithm = "Bresenham";
    String style = "Solid";

    public CircleDrawer(String algorithm, String style) {
        this.algorithm = algorithm;
        this.style = style;
        canvas = new BufferedImage(600, 600, BufferedImage.TYPE_INT_RGB);
        clearCanvas();
        drawCircle(centerX, centerY, radius, style, algorithm);
    }

    public void clearCanvas() {
        Graphics2D g = canvas.createGraphics();
        g.setColor(Color.WHITE);
        g.fillRect(0, 0, 600, 600);
    }

    public void plotCirclePoints(int xc, int yc, int x, int y, Color color, int step) {
        // Step controls dotted/dashed style
        if (step % getSkipFactor(style) != 0) return;

        int[] dx = {x, -x, x, -x, y, -y, y, -y};
        int[] dy = {y, y, -y, -y, x, x, -x, -x};
        for (int i = 0; i < 8; i++) {
            int px = xc + dx[i];
            int py = yc + dy[i];
            if (px >= 0 && py >= 0 && px < 600 && py < 600) {
                canvas.setRGB(px, py, color.getRGB());
            }
        }
    }

    public int getSkipFactor(String style) {
        switch (style.toLowerCase()) {
            case "dotted":
                return 4;
            case "dashed":
                return 2;
            default:
                return 1; // solid
        }
    }
}
```




```
public void drawCircle(int xc, int yc, int r, String style, String algorithm) {
    switch (algorithm.toLowerCase()) {
        case "dda":
            drawCircleDDA(xc, yc, r, style);
            break;
        case "midpoint":
            drawCircleMidpoint(xc, yc, r, style);
            break;
        default:
            drawCircleBresenham(xc, yc, r, style);
    }
}
```

```
public void drawCircleBresenham(int xc, int yc, int r, String style) {
    int x = 0, y = r;
    int d = 3 - 2 * r;
    int step = 0;
    while (x <= y) {
        plotCirclePoints(xc, yc, x, y, Color.BLUE, step++);
        if (d < 0)
            d = d + 4 * x + 6;
        else {
            d = d + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}
```

```
public void drawCircleMidpoint(int xc, int yc, int r, String style) {
    int x = 0, y = r;
    int p = 1 - r;
    int step = 0;
    while (x <= y) {
        plotCirclePoints(xc, yc, x, y, Color.RED, step++);
        if (p < 0)
            p += 2 * x + 3;
        else {
            p += 2 * (x - y) + 5;
            y--;
        }
        x++;
    }
}
```

```
public void drawCircleDDA(int xc, int yc, int r, String style) {
    double angle = 0;
    int step = 0;
    while (angle < 360) {
        double rad = Math.toRadians(angle);
        int x = (int) Math.round(xc + r * Math.cos(rad));
        int y = (int) Math.round(yc + r * Math.sin(rad));
    }
}
```



```
        if (step % getSkipFactor(style) == 0 && x >= 0 && y >= 0 && x < 600 && y < 600)
            canvas.setRGB(x, y, Color.MAGENTA.getRGB());
        angle += 0.5;
        step++;
    }
}

@Override
protected void paintComponent(Graphics g) {
    g.drawImage(canvas, 0, 0, null);
}

// Menu-driven main function
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String algoChoice = "Bresenham", styleChoice = "Solid";

    System.out.println("=== Circle Drawing Program ===");
    System.out.println("1. DDA\n2. Bresenham\n3. Midpoint");
    System.out.print("Select algorithm: ");
    int algo = sc.nextInt();
    switch (algo) {
        case 1: algoChoice = "DDA"; break;
        case 2: algoChoice = "Bresenham"; break;
        case 3: algoChoice = "Midpoint"; break;
    }

    System.out.println("\nChoose style:\n1. Solid\n2. Dotted\n3. Dashed");
    System.out.print("Select style: ");
    int style = sc.nextInt();
    switch (style) {
        case 1: styleChoice = "Solid"; break;
        case 2: styleChoice = "Dotted"; break;
        case 3: styleChoice = "Dashed"; break;
    }

    JFrame frame = new JFrame("Circle Drawing - " + algoChoice + " [" + styleChoice + "]");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(620, 640);
    frame.add(new CircleDrawer(algoChoice, styleChoice));
    frame.setVisible(true);
}
}
```



Write a menu driven program in C/C++/Java to draw a concave polygon and fill it with the desired color using the scan fill algorithm; flood fill and seed fill algorithms.

```
import javax.swing.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class PolygonFiller extends JPanel {
    BufferedImage canvas;
    Color fillColor = Color.RED;
    String fillMethod = "scan";

    int[] xPoints = { 100, 200, 150, 250, 300, 200 };
    int[] yPoints = { 100, 100, 200, 200, 300, 300 };
    int nPoints = xPoints.length;

    public PolygonFiller(String fillMethod) {
        this.fillMethod = fillMethod.toLowerCase();
        canvas = new BufferedImage(600, 600, BufferedImage.TYPE_INT_RGB);
        clearCanvas();
        drawPolygonOutline();
        if (this.fillMethod.equals("scan")) {
            scanlineFill(xPoints, yPoints, nPoints, fillColor);
        } else if (this.fillMethod.equals("flood")) {
            floodFill(160, 150, Color.BLACK.getRGB(), fillColor.getRGB());
        } else if (this.fillMethod.equals("seed")) {
            seedFill(160, 150, Color.BLACK.getRGB(), fillColor.getRGB());
        }
    }

    public void clearCanvas() {
        Graphics2D g = canvas.createGraphics();
        g.setColor(Color.WHITE);
        g.fillRect(0, 0, 600, 600);
    }

    public void drawPolygonOutline() {
        Graphics2D g = canvas.createGraphics();
        g.setColor(Color.BLACK);
        g.drawPolygon(xPoints, yPoints, nPoints);
    }

    // Scanline fill algorithm
    public void scanlineFill(int[] x, int[] y, int n, Color fillColor) {
        int[] nodes = new int[n];
        for (int yScan = 0; yScan < canvas.getHeight(); yScan++) {
            int count = 0;
            for (int i = 0; i < n; i++) {
```



```
int j = (i + 1) % n;
if ((y[i] < yScan && y[j] >= yScan) || (y[j] < yScan && y[i] >= yScan)) {
    nodes[count++] = x[i] + (yScan - y[i]) * (x[j] - x[i]) / (y[j] - y[i]);
}
}
java.util.Arrays.sort(nodes, 0, count);
for (int i = 0; i < count - 1; i += 2) {
    for (int xScan = nodes[i]; xScan < nodes[i + 1]; xScan++) {
        if (xScan >= 0 && xScan < canvas.getWidth())
            canvas.setRGB(xScan, yScan, fillColor.getRGB());
    }
}
}
```

// Recursive Flood Fill (4-connected)

```
public void floodFill(int x, int y, int oldColor, int newColor) {
    if (x < 0 || y < 0 || x >= canvas.getWidth() || y >= canvas.getHeight())
        return;
    if (canvas.getRGB(x, y) != oldColor || canvas.getRGB(x, y) == newColor)
        return;

    canvas.setRGB(x, y, newColor);
    floodFill(x + 1, y, oldColor, newColor);
    floodFill(x - 1, y, oldColor, newColor);
    floodFill(x, y + 1, oldColor, newColor);
    floodFill(x, y - 1, oldColor, newColor);
}
```

// Iterative Seed Fill (Queue based 4-connected)

```
public void seedFill(int x, int y, int oldColor, int newColor) {
    if (oldColor == newColor)
        return;

    Queue<Point> queue = new LinkedList<>();
    queue.add(new Point(x, y));

    while (!queue.isEmpty()) {
        Point p = queue.poll();
        int px = p.x, py = p.y;

        if (px < 0 || py < 0 || px >= canvas.getWidth() || py >= canvas.getHeight())
            continue;
        if (canvas.getRGB(px, py) != oldColor)
            continue;

        canvas.setRGB(px, py, newColor);

        queue.add(new Point(px + 1, py));
        queue.add(new Point(px - 1, py));
        queue.add(new Point(px, py + 1));
        queue.add(new Point(px, py - 1));
    }
}
```



```
}

@Override
protected void paintComponent(Graphics g) {
    g.drawImage(canvas, 0, 0, null);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("==== Concave Polygon Fill Menu =====");
    System.out.println("1. Scanline Fill");
    System.out.println("2. Flood Fill");
    System.out.println("3. Seed Fill");
    System.out.print("Choose option: ");
    int choice = sc.nextInt();

    String method;
    switch (choice) {
        case 1:
            method = "scan";
            break;
        case 2:
            method = "flood";
            break;
        case 3:
            method = "seed";
            break;
        default:
            method = "scan";
            break;
    }

    JFrame frame = new JFrame("Polygon Fill - " + method.toUpperCase());
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(620, 640);
    frame.add(new PolygonFiller(method));
    frame.setVisible(true);
}
}
```



Write a program to implement the Sutherland-Hodgeman algorithm for clipping any polygon. Provide the vertices of the polygon to be clipped and the pattern of clipping interactively

```
import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class Point {
    int x, y;
    Point(int a, int b) {
        x = a;
        y = b;
    }
}

public class SutherlandHodgmanClipping extends JPanel {
    List<Point> originalPolygon, clippedPolygon;
    int xmin, ymin, xmax, ymax;

    public SutherlandHodgmanClipping(List<Point> polygon, int xmin, int ymin, int xmax, int ymax) {
        this.originalPolygon = polygon;
        this.xmin = xmin;
        this.ymin = ymin;
        this.xmax = xmax;
        this.ymax = ymax;

        clippedPolygon = clipPolygon(originalPolygon, xmin, ymin, xmax, ymax);
    }

    // Clipping method using Sutherland-Hodgman
    private List<Point> clipPolygon(List<Point> polygon, int xmin, int ymin, int xmax, int ymax) {
        polygon = clipLeft(polygon, xmin);
        polygon = clipRight(polygon, xmax);
        polygon = clipBottom(polygon, ymin);
        polygon = clipTop(polygon, ymax);
        return polygon;
    }

    // Clipping boundaries
    private List<Point> clipLeft(List<Point> poly, int xmin) {
        return clip(poly, xmin, true, true);
    }

    private List<Point> clipRight(List<Point> poly, int xmax) {
        return clip(poly, xmax, true, false);
    }

    private List<Point> clipBottom(List<Point> poly, int ymin) {
        return clip(poly, ymin, false, true);
    }
```



```
}

private List<Point> clipTop(List<Point> poly, int ymax) {
    return clip(poly, ymax, false, false);
}

private List<Point> clip(List<Point> poly, int bound, boolean vertical, boolean insideIfLess) {
    List<Point> newPoly = new ArrayList<>();
    int n = poly.size();

    for (int i = 0; i < n; i++) {
        Point curr = poly.get(i);
        Point prev = poly.get((i + n - 1) % n);

        int c = vertical ? curr.x : curr.y;
        int p = vertical ? prev.x : prev.y;

        boolean currIn = insideIfLess ? c >= bound : c <= bound;
        boolean prevIn = insideIfLess ? p >= bound : p <= bound;

        if (prevIn && currIn) {
            newPoly.add(curr);
        } else if (prevIn && !currIn) {
            newPoly.add(intersect(prev, curr, bound, vertical));
        } else if (!prevIn && currIn) {
            newPoly.add(intersect(prev, curr, bound, vertical));
            newPoly.add(curr);
        }
    }
    return newPoly;
}

// Intersection point between line and clipping boundary
private Point intersect(Point a, Point b, int bound, boolean vertical) {
    float x, y;
    float dx = b.x - a.x;
    float dy = b.y - a.y;

    if (vertical) {
        x = bound;
        y = a.y + dy * (bound - a.x) / dx;
    } else {
        y = bound;
        x = a.x + dx * (bound - a.y) / dy;
    }

    return new Point(Math.round(x), Math.round(y));
}

// Draw method
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
}
```



```
// Draw original polygon (gray)
g.setColor(Color.LIGHT_GRAY);
drawPolygon(g, originalPolygon);

// Draw clipping rectangle (black)
g.setColor(Color.BLACK);
g.drawRect(xmin, ymin, xmax - xmin, ymax - ymin);

// Draw clipped polygon (red)
g.setColor(Color.RED);
drawPolygon(g, clippedPolygon);
}

private void drawPolygon(Graphics g, List<Point> poly) {
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        Point a = poly.get(i);
        Point b = poly.get((i + 1) % n);
        g.drawLine(a.x, a.y, b.x, b.y);
    }
}

// Entry point
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of polygon vertices: ");
    int n = sc.nextInt();
    List<Point> polygon = new ArrayList<>();

    for (int i = 0; i < n; i++) {
        System.out.print("Enter x y for vertex " + (i + 1) + ": ");
        int x = sc.nextInt();
        int y = sc.nextInt();
        polygon.add(new Point(x, y));
    }

    System.out.print("Enter clipping rectangle xmin ymin xmax ymax: ");
    int xmin = sc.nextInt();
    int ymin = sc.nextInt();
    int xmax = sc.nextInt();
    int ymax = sc.nextInt();

    JFrame frame = new JFrame("Sutherland-Hodgman Polygon Clipping");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(800, 600);
    frame.add(new SutherlandHodgmanClipping(polygon, xmin, ymin, xmax, ymax));
    frame.setVisible(true);
}
}
```

Write a C/C++/Java program to implement the Cohen-Sutherland line clipping algorithm.



```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class CohenSutherlandClipping extends JPanel {
    // Region codes
    static final int INSIDE = 0; // 0000
    static final int LEFT = 1; // 0001
    static final int RIGHT = 2; // 0010
    static final int BOTTOM = 4; // 0100
    static final int TOP = 8; // 1000

    int x1, y1, x2, y2; // Line endpoints
    int xmin, ymin, xmax, ymax; // Clipping window
    int[] clippedLine = new int[4];

    public CohenSutherlandClipping(int x1, int y1, int x2, int y2, int xmin, int ymin, int xmax, int ymax) {
        this.x1 = x1; this.y1 = y1;
        this.x2 = x2; this.y2 = y2;
        this.xmin = xmin; this.ymin = ymin;
        this.xmax = xmax; this.ymax = ymax;

        boolean visible = cohenSutherlandClip();
        if (!visible) clippedLine = null;
    }

    // Compute region code for a point (x, y)
    int computeCode(int x, int y) {
        int code = INSIDE;

        if (x < xmin)
            code |= LEFT;
        else if (x > xmax)
            code |= RIGHT;
        if (y < ymin)
            code |= BOTTOM;
        else if (y > ymax)
            code |= TOP;

        return code;
    }

    // Cohen-Sutherland clipping algorithm
    boolean cohenSutherlandClip() {
        int code1 = computeCode(x1, y1);
        int code2 = computeCode(x2, y2);
        boolean accept = false;

        while (true) {
            if ((code1 | code2) == 0) {
                // Trivially accepted
                accept = true;
                clippedLine[0] = x1;
            }
        }
    }
}
```



```
        clippedLine[1] = y1;
        clippedLine[2] = x2;
        clippedLine[3] = y2;
        break;
    } else if ((code1 & code2) != 0) {
        // Trivially rejected
        break;
    } else {
        int codeOut = (code1 != 0) ? code1 : code2;
        int x = 0, y = 0;

        if ((codeOut & TOP) != 0) {
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
            y = ymax;
        } else if ((codeOut & BOTTOM) != 0) {
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
            y = ymin;
        } else if ((codeOut & RIGHT) != 0) {
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
            x = xmax;
        } else if ((codeOut & LEFT) != 0) {
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
            x = xmin;
        }

        if (codeOut == code1) {
            x1 = x;
            y1 = y;
            code1 = computeCode(x1, y1);
        } else {
            x2 = x;
            y2 = y;
            code2 = computeCode(x2, y2);
        }
    }
}

return accept;
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);

    // Draw clipping rectangle
    g.setColor(Color.BLACK);
    g.drawRect(xmin, ymin, xmax - xmin, ymax - ymin);

    // Draw original line
    g.setColor(Color.GRAY);
    g.drawLine(this.x1, this.y1, this.x2, this.y2);

    // Draw clipped line
```



```
        if (clippedLine != null) {
            g.setColor(Color.RED);
            g.drawLine(clippedLine[0], clippedLine[1], clippedLine[2], clippedLine[3]);
        } else {
            g.setColor(Color.BLUE);
            g.drawString("Line completely outside and rejected", 20, 20);
        }
    }

// Main method for user input
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter line endpoints (x1 y1 x2 y2): ");
    int x1 = sc.nextInt(), y1 = sc.nextInt(), x2 = sc.nextInt(), y2 = sc.nextInt();

    System.out.println("Enter clipping window (xmin ymin xmax ymax): ");
    int xmin = sc.nextInt(), ymin = sc.nextInt(), xmax = sc.nextInt(), ymax = sc.nextInt();

    JFrame frame = new JFrame("Cohen-Sutherland Line Clipping");
    frame.setSize(800, 600);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.add(new CohenSutherlandClipping(x1, y1, x2, y2, xmin, ymin, xmax, ymax));
    frame.setVisible(true);
}
```



Write a C/C++/Java program to implement translation, rotation, shear and scaling transformations on a 2D object about X axis, Y axis

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class Transform2D extends JPanel {
    double[][] points;
    double[][] transformedPoints;
    int n;

    public Transform2D(double[][] points, double[][] matrix) {
        this.points = points;
        this.n = points.length;
        this.transformedPoints = applyTransformation(points, matrix);
    }

    // Apply matrix transformation
    public double[][] applyTransformation(double[][] pts, double[][] mat) {
        double[][] result = new double[n][2];
        for (int i = 0; i < n; i++) {
            double x = pts[i][0], y = pts[i][1];
            result[i][0] = x * mat[0][0] + y * mat[0][1] + mat[0][2];
            result[i][1] = x * mat[1][0] + y * mat[1][1] + mat[1][2];
        }
        return result;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        // Draw original polygon in blue
        g.setColor(Color.BLUE);
        drawPolygon(g, points);

        // Draw transformed polygon in red
        g.setColor(Color.RED);
        drawPolygon(g, transformedPoints);
    }

    private void drawPolygon(Graphics g, double[][] poly) {
        for (int i = 0; i < poly.length; i++) {
            int x1 = (int) poly[i][0], y1 = (int) poly[i][1];
            int j = (i + 1) % poly.length;
            int x2 = (int) poly[j][0], y2 = (int) poly[j][1];
            g.drawLine(x1, y1, x2, y2);
        }
    }
}
```



```
// Helper to get transformation matrix
public static double[][] getTransformationMatrix(String type, Scanner sc) {
    switch (type.toLowerCase()) {
        case "translate":
            System.out.print("Enter dx and dy: ");
            double dx = sc.nextDouble(), dy = sc.nextDouble();
            return new double[][]{
                {1, 0, dx},
                {0, 1, dy},
                {0, 0, 1}
            };
        case "scale":
            System.out.print("Enter sx and sy: ");
            double sx = sc.nextDouble(), sy = sc.nextDouble();
            return new double[][]{
                {sx, 0, 0},
                {0, sy, 0},
                {0, 0, 1}
            };
        case "rotate":
            System.out.print("Enter angle in degrees: ");
            double angle = Math.toRadians(sc.nextDouble());
            return new double[][]{
                {Math.cos(angle), -Math.sin(angle), 0},
                {Math.sin(angle), Math.cos(angle), 0},
                {0, 0, 1}
            };
        case "shearx":
            System.out.print("Enter shear factor shx: ");
            double shx = sc.nextDouble();
            return new double[][]{
                {1, shx, 0},
                {0, 1, 0},
                {0, 0, 1}
            };
        case "sheary":
            System.out.print("Enter shear factor shy: ");
            double shy = sc.nextDouble();
            return new double[][]{
                {1, 0, 0},
                {shy, 1, 0},
                {0, 0, 1}
            };
        default:
            return new double[][]{
                {1, 0, 0},
                {0, 1, 0},
                {0, 0, 1}
            };
    }
}
```

// Main method



```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Define polygon
    double[][] polygon = {
        {100, 100},
        {200, 100},
        {200, 200},
        {100, 200}
    };

    System.out.println("Choose transformation:");
    System.out.println("1. Translate");
    System.out.println("2. Scale");
    System.out.println("3. Rotate");
    System.out.println("4. Shear X");
    System.out.println("5. Shear Y");
    System.out.print("Enter choice (1-5): ");
    int choice = sc.nextInt();

    String type = switch (choice) {
        case 1 -> "translate";
        case 2 -> "scale";
        case 3 -> "rotate";
        case 4 -> "shearx";
        case 5 -> "sheary";
        default -> "";
    };

    double[][] matrix = getTransformationMatrix(type, sc);

    // Create window and draw
    JFrame frame = new JFrame("2D Transformation - " + type.toUpperCase());
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(600, 600);
    frame.add(new Transform2D(polygon, matrix));
    frame.setVisible(true);
}
}
```

Option	Transformation	Parameters
1	Translate	dx, dy
2	Scale	sx, sy
3	Rotate	angle (degrees)
4	Shear in X	shear factor shx
5	Shear in Y	shear factor shy



Write C/C++/Java program to implement translation, sheer, rotation and scaling transformations on equilateral triangle and rhombus.

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class ShapeTransformations extends JPanel {
    double[][] originalPoints;
    double[][] transformedPoints;

    public ShapeTransformations(double[][] shape, double[][] transformationMatrix) {
        this.originalPoints = shape;
        this.transformedPoints = applyTransformation(shape, transformationMatrix);
    }

    // Apply 2D Transformation
    public double[][] applyTransformation(double[][] points, double[][] mat) {
        double[][] result = new double[points.length][2];
        for (int i = 0; i < points.length; i++) {
            double x = points[i][0];
            double y = points[i][1];
            result[i][0] = x * mat[0][0] + y * mat[0][1] + mat[0][2];
            result[i][1] = x * mat[1][0] + y * mat[1][1] + mat[1][2];
        }
        return result;
    }

    // Drawing
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawShape(g, originalPoints, Color.BLUE); // Original
        drawShape(g, transformedPoints, Color.RED); // Transformed
    }

    private void drawShape(Graphics g, double[][] shape, Color color) {
        g.setColor(color);
        for (int i = 0; i < shape.length; i++) {
            int x1 = (int) shape[i][0];
            int y1 = (int) shape[i][1];
            int x2 = (int) shape[(i + 1) % shape.length][0];
            int y2 = (int) shape[(i + 1) % shape.length][1];
            g.drawLine(x1, y1, x2, y2);
        }
    }

    // Main Method
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Choose shape:\n1. Equilateral Triangle\n2. Rhombus");
```



```
int shapeChoice = sc.nextInt();

double[][] shape;
if (shapeChoice == 1) {
    shape = new double[][]{
        {200, 200},
        {300, 200},
        {250, 115}
    };
} else {
    shape = new double[][]{
        {200, 200},
        {250, 150},
        {300, 200},
        {250, 250}
    };
}

System.out.println("Choose transformation:");
System.out.println("1. Translation\n2. Scaling\n3. Rotation\n4. Shear X\n5. Shear Y");
int transformChoice = sc.nextInt();

double[][] mat = switch (transformChoice) {
    case 1 -> {
        System.out.print("Enter dx and dy: ");
        double dx = sc.nextDouble(), dy = sc.nextDouble();
        yield new double[][]{
            {1, 0, dx},
            {0, 1, dy},
            {0, 0, 1}
        };
    }
    case 2 -> {
        System.out.print("Enter sx and sy: ");
        double sx = sc.nextDouble(), sy = sc.nextDouble();
        yield new double[][]{
            {sx, 0, 0},
            {0, sy, 0},
            {0, 0, 1}
        };
    }
    case 3 -> {
        System.out.print("Enter angle (degrees): ");
        double angle = Math.toRadians(sc.nextDouble());
        yield new double[][]{
            {Math.cos(angle), -Math.sin(angle), 0},
            {Math.sin(angle), Math.cos(angle), 0},
            {0, 0, 1}
        };
    }
    case 4 -> {
        System.out.print("Enter shear X value: ");
        double shx = sc.nextDouble();
```




```
        yield new double[][]{
            {1, shx, 0},
            {0, 1, 0},
            {0, 0, 1}
        };
    }
    case 5 -> {
        System.out.print("Enter shear Y value: ");
        double shy = sc.nextDouble();
        yield new double[][]{
            {1, 0, 0},
            {shy, 1, 0},
            {0, 0, 1}
        };
    }
    default -> new double[][]{
        {1, 0, 0},
        {0, 1, 0},
        {0, 0, 1}
    };
};

JFrame frame = new JFrame("2D Transformations - Triangle/Rhombus");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(600, 600);
frame.add(new ShapeTransformations(shape, mat));
frame.setVisible(true);
}
}
```

Choose shape:

2

Choose transformation:

4

Enter shear X value: 0.5



Write a C/C++/Java program to implement rotation of a 2D object about X axis and an arbitrary point.

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class Rotation2D extends JPanel {
    double[][] originalPoints;
    double[][] rotatedPoints;

    public Rotation2D(double[][] shape, double[][] transformMatrix) {
        this.originalPoints = shape;
        this.rotatedPoints = applyTransformation(shape, transformMatrix);
    }

    private double[][] applyTransformation(double[][] points, double[][] matrix) {
        double[][] result = new double[points.length][2];
        for (int i = 0; i < points.length; i++) {
            double x = points[i][0];
            double y = points[i][1];
            result[i][0] = x * matrix[0][0] + y * matrix[0][1] + matrix[0][2];
            result[i][1] = x * matrix[1][0] + y * matrix[1][1] + matrix[1][2];
        }
        return result;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawShape(g, originalPoints, Color.BLUE);
        drawShape(g, rotatedPoints, Color.RED);
    }

    private void drawShape(Graphics g, double[][] shape, Color color) {
        g.setColor(color);
        for (int i = 0; i < shape.length; i++) {
            int x1 = (int) shape[i][0], y1 = (int) shape[i][1];
            int x2 = (int) shape[(i + 1) % shape.length][0];
            int y2 = (int) shape[(i + 1) % shape.length][1];
            g.drawLine(x1, y1, x2, y2);
        }
    }

    // Generate rotation matrix about arbitrary point (xr, yr)
    public static double[][] rotationAboutPointMatrix(double angleDeg, double xr, double yr) {
        double angleRad = Math.toRadians(angleDeg);
        double cos = Math.cos(angleRad), sin = Math.sin(angleRad);

        return new double[][] {
            {cos, -sin, xr * (1 - cos) + yr * sin},
            {sin, cos, yr * (1 - cos) - xr * sin},
            {0, 0, 1}
        };
    }
}
```



```
}

// Main method
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Example shape: Rectangle
    double[][] shape = {
        {200, 200},
        {300, 200},
        {300, 300},
        {200, 300}
    };

    System.out.println("Choose rotation type:");
    System.out.println("1. About X-axis (flip Y)");
    System.out.println("2. About arbitrary point");
    int choice = sc.nextInt();

    double[][] matrix;

    if (choice == 1) {
        matrix = new double[][] {
            {1, 0, 0},
            {0, -1, 600}, // flip Y over X-axis (assuming height=600)
            {0, 0, 1}
        };
    } else {
        System.out.print("Enter rotation angle (degrees): ");
        double angle = sc.nextDouble();
        System.out.print("Enter X and Y coordinates of rotation point: ");
        double xr = sc.nextDouble(), yr = sc.nextDouble();

        matrix = rotationAboutPointMatrix(angle, xr, yr);
    }

    JFrame frame = new JFrame("2D Rotation - X-axis or Arbitrary Point");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(800, 600);
    frame.add(new Rotation2D(shape, matrix));
    frame.setVisible(true);
}
}
```

Choose rotation type: 1
Choose rotation type: 2
Enter rotation angle (degrees): 45
Enter X and Y coordinates of rotation point: 250 250



Mini-Project using maximum features of Object-Oriented programming to develop solutions for any one a1 Banking system having the following operations: a. Create an account b. Deposit money c. Withdraw money d. Honor daily withdrawal limit e. Check the balance f. Display Account information. g. Passbook Print (from to)

```
import java.util.*;
import java.time.*;
import java.time.format.DateTimeFormatter;

class Transaction {
    LocalDate date;
    String type;
    double amount;
    double balance;

    public Transaction(String type, double amount, double balance) {
        this.date = LocalDate.now();
        this.type = type;
        this.amount = amount;
        this.balance = balance;
    }

    public String toString() {
        return date + " - " + type + ": ₹" + amount + " | Balance: ₹" + balance;
    }
}

class Account {
    private static int idCounter = 1001;
    private final String accountNumber;
    private String name;
    private double balance;
    private double dailyWithdrawn;
    private static final double DAILY_LIMIT = 20000.0;
    private List<Transaction> transactions;

    public Account(String name, double initialDeposit) {
        this.accountNumber = "ACC" + idCounter++;
        this.name = name;
        this.balance = initialDeposit;
        this.dailyWithdrawn = 0;
        this.transactions = new ArrayList<>();
        transactions.add(new Transaction("Deposit", initialDeposit, balance));
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("✗Invalid deposit amount.");
            return;
        }
    }
}
```



```
}
balance += amount;
transactions.add(new Transaction("Deposit", amount, balance));
System.out.println("✓ Deposited ₹" + amount);
}

public void withdraw(double amount) {
    if (amount <= 0 || amount > balance) {
        System.out.println("✗ Invalid withdrawal.");
    } else if (dailyWithdrawn + amount > DAILY_LIMIT) {
        System.out.println("✗ Daily withdrawal limit exceeded.");
    } else {
        balance -= amount;
        dailyWithdrawn += amount;
        transactions.add(new Transaction("Withdraw", amount, balance));
        System.out.println("✓ Withdrawn ₹" + amount);
    }
}

public void checkBalance() {
    System.out.println("□ Current balance: ₹" + balance);
}

public void displayInfo() {
    System.out.println("\n□ Account Number: " + accountNumber);
    System.out.println("□ Account Holder: " + name);
    System.out.println("□ Balance: ₹" + balance);
    System.out.println("□ Today's Withdrawal: ₹" + dailyWithdrawn + " / ₹" + DAILY_LIMIT);
}

public void printPassbook(LocalDate from, LocalDate to) {
    System.out.println("\n□ Passbook for Account: " + accountNumber);
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");

    for (Transaction t : transactions) {
        if ((t.date.isEqual(from) || t.date.isAfter(from)) &&
            (t.date.isEqual(to) || t.date.isBefore(to))) {
            System.out.println(t);
        }
    }
}

public class BankSystem {
    static Scanner sc = new Scanner(System.in);
    static Map<String, Account> accounts = new HashMap<>();

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n===== BANK MENU =====");
            System.out.println("1. Create Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
        }
    }
}
```



```
System.out.println("4. Check Balance");
System.out.println("5. Account Info");
System.out.println("6. Print Passbook");
System.out.println("7. Exit");
System.out.print("Choose option: ");
int ch = sc.nextInt();

switch (ch) {
    case 1:
        createAccount();
        break;
    case 2:
        depositMoney();
        break;
    case 3:
        withdrawMoney();
        break;
    case 4:
        checkBalance();
        break;
    case 5:
        displayAccountInfo();
        break;
    case 6:
        printPassbook();
        break;
    case 7:
        System.out.println("☐ Thank you for using the banking system.");
        return;
    default:
        System.out.println("✗ Invalid choice.");
        break;
}
}
}

static void createAccount() {
    sc.nextLine();
    System.out.print("Enter name: ");
    String name = sc.nextLine();
    System.out.print("Initial deposit: ");
    double deposit = sc.nextDouble();
    Account acc = new Account(name, deposit);
    accounts.put(acc.getAccountNumber(), acc);
    System.out.println("✓ Account created. Account Number: " + acc.getAccountNumber());
}

static Account getAccount() {
    sc.nextLine();
    System.out.print("Enter account number: ");
    String accNo = sc.nextLine();
    Account acc = accounts.get(accNo);
    if (acc == null) {
```



```
        System.out.println("✗Account not found.");
    }
    return acc;
}

static void depositMoney() {
    Account acc = getAccount();
    if (acc != null) {
        System.out.print("Enter deposit amount: ");
        acc.deposit(sc.nextDouble());
    }
}

static void withdrawMoney() {
    Account acc = getAccount();
    if (acc != null) {
        System.out.print("Enter withdrawal amount: ");
        acc.withdraw(sc.nextDouble());
    }
}

static void checkBalance() {
    Account acc = getAccount();
    if (acc != null)
        acc.checkBalance();
}

static void displayAccountInfo() {
    Account acc = getAccount();
    if (acc != null)
        acc.displayInfo();
}

static void printPassbook() {
    Account acc = getAccount();
    if (acc != null) {
        System.out.print("Enter start date (yyyy-mm-dd): ");
        LocalDate from = LocalDate.parse(sc.next());
        System.out.print("Enter end date (yyyy-mm-dd): ");
        LocalDate to = LocalDate.parse(sc.next());
        acc.printPassbook(from, to);
    }
}
}
```

===== BANK MENU =====

1. Create Account
2. Deposit
3. Withdraw
4. Check Balance
5. Account Info
6. Print Passbook
7. Exit



Choose option: 1
Enter name: Riya
Initial deposit: 10000
✓ Account created. Account Number: ACC1001

===== BANK MENU =====
Choose option: 2
Enter account number: ACC1001
Enter deposit amount: 5000
✓ Deposited ₹5000

Inventory management system having the following operations: a. List of all products b. Display individual product information c. Purchase d. Shipping e. Balance stock f. Loss and Profit calculation. g. Purchase Report (from to)

```
import java.time.LocalDate;  
import java.util.*;
```

```
class Transaction {  
    LocalDate date;  
    int quantity;  
    double pricePerUnit;  
  
    public Transaction(int quantity, double pricePerUnit) {  
        this.date = LocalDate.now();  
        this.quantity = quantity;  
        this.pricePerUnit = pricePerUnit;  
    }  
  
    public String toString() {  
        return date + " | Qty: " + quantity + " | ₹" + pricePerUnit;  
    }  
}
```

```
class Product {  
    String id;  
    String name;  
    int stock;  
    double costPrice;  
    double sellingPrice;  
    List<Transaction> purchases;  
    List<Transaction> sales;  
  
    public Product(String id, String name, double costPrice, double sellingPrice) {  
        this.id = id;  
        this.name = name;  
        this.stock = 0;  
        this.costPrice = costPrice;  
        this.sellingPrice = sellingPrice;  
        this.purchases = new ArrayList<>();  
        this.sales = new ArrayList<>();  
    }  
}
```




```
public void purchase(int quantity) {
    stock += quantity;
    purchases.add(new Transaction(quantity, costPrice));
    System.out.println("✓ Purchased " + quantity + " units of " + name);
}

public void ship(int quantity) {
    if (quantity > stock) {
        System.out.println("✗ Not enough stock to ship.");
    } else {
        stock -= quantity;
        sales.add(new Transaction(quantity, sellingPrice));
        System.out.println("✓ Shipped " + quantity + " units of " + name);
    }
}

public void displayInfo() {
    System.out.println("\n□ Product ID: " + id);
    System.out.println("□ Name: " + name);
    System.out.println("□ Cost Price: ₹" + costPrice);
    System.out.println("□ Selling Price: ₹" + sellingPrice);
    System.out.println("□ Stock: " + stock);
}

public int getStock() {
    return stock;
}

public void reportPurchases(LocalDate from, LocalDate to) {
    System.out.println("\n□ Purchase Report for: " + name);
    for (Transaction t : purchases) {
        if (!t.date.isBefore(from) && !t.date.isAfter(to)) {
            System.out.println(t);
        }
    }
}

public void calculateProfitOrLoss() {
    double totalCost = 0, totalSales = 0;
    for (Transaction t : purchases) {
        totalCost += t.quantity * t.pricePerUnit;
    }
    for (Transaction t : sales) {
        totalSales += t.quantity * t.pricePerUnit;
    }

    double profit = totalSales - totalCost;
    System.out.println("\n□ Profit/Loss for " + name + ": ₹" + profit +
        (profit >= 0 ? " (Profit)" : " (Loss)"));
}
```



```
public class InventorySystem {
    static Scanner sc = new Scanner(System.in);
    static Map<String, Product> inventory = new HashMap<>();

    public static void main(String[] args) {
        initializeProducts();

        while (true) {
            System.out.println("\n===== INVENTORY MENU =====");
            System.out.println("1. List all products");
            System.out.println("2. Display product info");
            System.out.println("3. Purchase");
            System.out.println("4. Shipping");
            System.out.println("5. Balance stock");
            System.out.println("6. Profit/Loss Calculation");
            System.out.println("7. Purchase Report");
            System.out.println("8. Exit");
            System.out.print("Choose option: ");
            int ch = sc.nextInt();

            switch (ch) {
                case 1:
                    listProducts();
                    break;
                case 2:
                    displayProduct();
                    break;
                case 3:
                    purchaseProduct();
                    break;
                case 4:
                    shipProduct();
                    break;
                case 5:
                    checkStock();
                    break;
                case 6:
                    calculateProfitLoss();
                    break;
                case 7:
                    purchaseReport();
                    break;
                case 8:
                    System.out.println("✓ Thank you for using Inventory System.");
                    return;
                default:
                    System.out.println("✗ Invalid choice.");
                    break;
            }
        }
    }

    static void initializeProducts() {
```



```
inventory.put("P101", new Product("P101", "Laptop", 40000, 50000));
inventory.put("P102", new Product("P102", "Smartphone", 15000, 20000));
inventory.put("P103", new Product("P103", "Headphones", 1000, 1500));
}

static void listProducts() {
    System.out.println("\n Available Products:");
    for (Product p : inventory.values()) {
        System.out.println("\t" + p.id + " - " + p.name);
    }
}

static Product getProduct() {
    sc.nextLine(); // Consume leftover newline
    System.out.print("Enter Product ID: ");
    String pid = sc.nextLine();
    Product p = inventory.get(pid);
    if (p == null) {
        System.out.println("✗Product not found.");
    }
    return p;
}

static void displayProduct() {
    Product p = getProduct();
    if (p != null) p.displayInfo();
}

static void purchaseProduct() {
    Product p = getProduct();
    if (p != null) {
        System.out.print("Enter quantity to purchase: ");
        int qty = sc.nextInt();
        p.purchase(qty);
    }
}

static void shipProduct() {
    Product p = getProduct();
    if (p != null) {
        System.out.print("Enter quantity to ship: ");
        int qty = sc.nextInt();
        p.ship(qty);
    }
}

static void checkStock() {
    System.out.println("\n Stock Report:");
    for (Product p : inventory.values()) {
        System.out.println(p.name + ": " + p.getStock() + " units");
    }
}
```



```
static void calculateProfitLoss() {
    Product p = getProduct();
    if (p != null) {
        p.calculateProfitOrLoss();
    }
}

static void purchaseReport() {
    Product p = getProduct();
    if (p != null) {
        System.out.print("Enter start date (yyyy-mm-dd): ");
        LocalDate from = LocalDate.parse(sc.next());
        System.out.print("Enter end date (yyyy-mm-dd): ");
        LocalDate to = LocalDate.parse(sc.next());
        p.reportPurchases(from, to);
    }
}
```

===== INVENTORY MENU =====

1. List all products
2. Display product info
3. Purchase
4. Shipping
5. Balance stock
6. Profit/Loss Calculation
7. Purchase Report
8. Exit

Choose option: 1

- ☐ Available Products:
- ☐ P101 - Laptop
 - ☐ P102 - Smartphone
 - ☐ P103 – Headphones

All the function